

A Lean 4 library published with its proofs withheld

Every statement in clear, every proof term replaced by a commitment
and a machine-checked argument of knowledge of its opening

Aristotle (Harmonic)

Generated by `scripts/ExportLedger.lean` from the repository itself

Abstract

This document publishes a Lean 4 library without publishing its proofs. The library has 1570 declarations (924 theorems and 646 definitions) in 44 modules. Every statement is printed in clear, here and in the companion volume; not one proof term, and no fragment of one, appears in either document. What appears instead is, for each declaration, the SHA-256 of the printed statement, the SHA-256 of the canonical serialisation of its elaborated type, the SHA-256 of the withheld serialised proof term, a Pedersen commitment to that digest, and a Fiat–Shamir compiled Schnorr transcript proving knowledge of the opening of that commitment, bound to the declaration it belongs to. The withheld serialisations are 3798215 bytes over 318039 distinct kernel nodes. All 1570 transcripts were re-checked at generation time, and the entries fold into the ledger root `d6ae54acefd72ba15da13e06cf26b5cd9bf4c37baf98969308cacf976f771db8`.

The scheme is not merely described here: it is part of the library it publishes. The hash function, the modular exponentiation, the commitment, the argument and the Merkle tree are Lean definitions of `RequestProject/Publication/`, and their completeness, extractability, hiding, zero-knowledge and binding properties are theorems of the same library, proved with no `sorry`. Those theorems therefore appear in this ledger, committed to and withheld, like every other statement. Section 7 says precisely what this construction does *not* establish.

Contents

1	What this document is	2
2	The library	3
3	The publication scheme	3
3.1	Hashing	3
3.2	Arithmetic	3
3.3	The group and its scalars	3
3.4	Commitment and argument	3
3.5	Fiat–Shamir and the ledger entry	4
3.6	The ledger root	4
4	The published group	5

5	The declaration-local finite-group envelope	5
5.1	The invariant	5
5.2	The group sandwich	5
5.3	The local field prime	6
5.4	What the reader checks, and what it means	6
5.5	This ledger	7
6	The verification recipe	7
7	What is claimed, and what is not	7
8	Three entries in full	8
9	The library, module by module	10
10	Reproducing this document	10

1 What this document is

A formal library consists of statements and proofs. The statements are what one wants to read; the proof terms are large, unreadable, and — for a library one may wish to license, stage or embargo — the part one may not wish to hand over. This document is an experiment in publishing the first without the second, in such a way that the reader still gets a cryptographic object standing in the place of each withheld proof.

For every declaration of the library the ledger records

- its fully qualified name and its kind (theorem or definition);
- its statement, printed in clear by Lean’s own pretty printer;
- SHA-256 of that printed statement, so that the reader can check that the text in front of them is the text that was committed to;
- SHA-256 of the canonical serialisation of the elaborated type — the pretty-printer-independent form of the statement;
- SHA-256 of the canonical serialisation of the withheld proof term, its size in bytes, its number of distinct kernel nodes, and its axiom closure;
- a Pedersen commitment C to that digest, and a Schnorr transcript (t, c, z, w) proving knowledge of an opening of C , with the challenge c obtained by hashing the declaration’s identity together with C and t .

The complete ledger is the companion volume; three entries are reproduced in full in Section 8, and Section 9 gives the per-module summary.

2 The library

The published library is the repository this document was generated from: a development about finite labelled objects, the tower of constraints mined over them, the zero-knowledge forms of that tower, and a self-hosting holographic description of the whole system. Its 1570 published declarations are distributed over 44 modules as tabulated in Section 9.

The library builds with `lake build`, contains no `sorry`, and every declaration’s axiom closure is printed in its ledger entry. That build — not the transcripts below — is what establishes that the withheld proofs exist and are correct; see Section 7.

3 The publication scheme

Everything in this section is a definition or a theorem of the published library. Lean names are given so that each claim can be located in the ledger and, for a reader with the source, checked by the kernel.

3.1 Hashing

`Publication.Sha256.hashBytes` is SHA-256, written out in Lean following FIPS 180-4, and executable inside the kernel; `Publication.Sha256.hashHex` is its hexadecimal rendering. The standard test vectors `Sha256.hash_empty`, `Sha256.hash_abc` and `Sha256.hash_two_block` are theorems, decided by computation. Every digest in this document and in the companion volume was produced by this definition.

3.2 Arithmetic

`Publication.powMod` is square-and-multiply modular exponentiation, and `Publication.powMod_eq` proves

$$\text{powMod } a \ e \ m = a^e \bmod m,$$

so the numbers printed in the ledger are the mathematical ones and not an artefact of the implementation.

3.3 The group and its scalars

Work in a commutative group G and fix elements of exponent q , i.e. $g^q = h^q = 1$. Scalars are then the field $\mathbb{Z}/q$, acting by `zexp` $x \ a = x^{a.\text{val}}$. That this is an action of both the additive and the multiplicative structure is `Publication.zexp_add` and `Publication.zexp_mul`; the proofs use only $x^q = 1$.

3.4 Commitment and argument

Definition 3.1 (`Publication.Ledger.com`). The commitment to a digest m with blinding r is $C = \text{com}(m, r) = g^m h^r$.

Definition 3.2 (`Publication.Ledger.prove`, `Accepts`). With nonce (k, s) and challenge c , the prover publishes $t = g^k h^s$, $z = k + cm$ and $w = s + cr$. The verifier accepts iff

$$g^z h^w = t \cdot C^c.$$

Theorem 3.3 (completeness; `Ledger.completeness`). *The honest prover is accepted, for every digest, blinding, nonce and challenge.*

Theorem 3.4 (special soundness; `Ledger.special_soundness`). *Two accepting transcripts with the same first message and different challenges determine an opening of C : the explicit extractor `Ledger.extract`, applied to them, returns a pair (m, r) with $\text{com}(m, r) = C$. Knowledge of the committed digest is therefore computable from a prover by rewinding.*

Theorem 3.5 (hiding; `Ledger.commit_view_indep`, `Ledger.perfect_hiding`). *If g is a power of h , then for a uniformly distributed blinding the distribution of C is the same whatever digest is committed to. The published commitment therefore carries no information about the withheld proof term.*

Theorem 3.6 (zero knowledge; `Ledger.transcript_view_indep`). *For a uniformly distributed nonce the honest transcript has exactly the distribution produced by the simulator `Ledger.simulate`, which never sees the witness. This is an equality of probability distributions (PMF), not merely of supports.*

Theorem 3.7 (binding is discrete logarithm; `Ledger.binding_extract_dlog`). *Two different openings of one commitment yield an explicit discrete logarithm of h to base g . Binding is therefore exactly as hard as the discrete logarithm problem in the published group — and no harder, which at the parameters of Section 4 means: not hard at all.*

3.5 Fiat–Shamir and the ledger entry

The challenge is not chosen interactively but computed (`Certificate.challengeOf`) as

$$c = \text{SHA-256}(\text{harmonic} - \text{zk} - \text{ledger}/1 \parallel \text{name} \parallel \text{statement digest} \parallel \text{type digest} \parallel \text{env} \parallel \text{Com}(\text{envelope}) \parallel C \parallel t) \bmod q$$

Hashing the declaration’s identity into the challenge is what binds the transcript to *that* statement; hashing the commitment to its declaration-local envelope (Section 5) is what stops the local field parameter from being chosen after the challenge is known.

Theorem 3.8 (the exporter is complete; `Certificate.mkEntry_verifies`). *For every name, statement, type digest, term digest, size, axiom string, seed and envelope that passes its own check, the entry produced by `Certificate.mkEntry` passes `Certificate.Entry.verify`. Every entry in the companion volume is an instance of this theorem, and each was in addition re-checked by running the verifier at generation time: 1570 of 1570.*

3.6 The ledger root

Entry texts are hashed into the leaves of a perfect binary Merkle tree (`Publication.Merkle`) with domain separation between leaves and nodes. `Merkle.exists_proof` gives every leaf an inclusion proof that recomputes the root; `Merkle.verify_sound` shows conversely that an inclusion proof of the right length for anything that is *not* a leaf yields an explicit collision of the compression function. Under collision resistance of SHA-256 — an assumption, not a theorem — the single value

ledger root d6ae54acefd72ba15da13e06cf26b5cd9bf4c37baf98969308cacf976f771db8

therefore pins down the whole ledger.

4 The published group

modulus	$p = 2134403959322183$
subgroup order	$q = 1067201979661091$, with $p = 2q + 1$
generators	$g = 4, h = 9$
relation	$h^y = g$ with $y = 98140825630993$

Both primalities are theorems of the library, decided by computation (`Params.p_prime`, `Params.q_prime`), as are $g^q = h^q = 1$ (`Params.g_pow_q`, `Params.h_pow_q`) and $h^y = g$ (`Params.h_pow_y`). The last is what the hiding theorem needs, and it is published deliberately: at this size the discrete logarithm is computable, and pretending otherwise would be dishonest.

These are demonstration-scale parameters — 51 bits — chosen so that every arithmetical claim about them is machine-checked rather than assumed. Nothing in the scheme depends on their size: the theorems of `RequestProject/Publication/Ledger.lean` are stated for an arbitrary commutative group of prime exponent, and substituting a cryptographic-size group changes no proof. What such a substitution would cost is the machine-checked primality of the parameters, which at 2048 bits is out of reach of the kernel’s arithmetic by trial division.

5 The declaration-local finite-group envelope

Every entry carries, besides its digests and its transcript, a *declaration-local finite-group envelope*: a pair of finite groups whose orders bracket a complexity invariant of the declaration, and a field prime derived from that bracket. Nothing in it is chosen; everything in it is checked.

5.1 The invariant

For a declaration d the exporter measures, off the fully elaborated declaration,

$$\kappa(d) = (|\text{DAG}(\tau_d)|, |\text{DAG}(v_d)|, |\text{Deps}(d)|, \text{maxUniverse}(d), \text{checkSteps}(d)),$$

the DAG sizes of the elaborated type and of the withheld value, the number of constants the declaration mentions, the largest universe depth occurring in it, and the size of the environment fragment a kernel recheck traverses (the two DAG sizes plus the DAG size of the type of every constant mentioned). All five are relative to the canonical serialiser of `scripts/ExportLedger.lean`, which is the same serialiser whose output is hashed into the withheld-term digest. The scalarization is the sum, $N(d) = \tau + \nu + \delta + v + \kappa$ (`Envelope.Kappa.N`), monotone in each coordinate.

5.2 The group sandwich

The envelope proper is the factorial sandwich: the index n_d with

$$n_d! \leq N(d) < (n_d + 1)!, \quad G_d^- = S_{n_d}, \quad G_d^+ = S_{n_d+1}.$$

Theorem 5.1 (existence, uniqueness, orders and chain). *Every positive scale is sandwiched (`Envelope.factSandwich_exists`) and by exactly one index (`Envelope.factSandwich_unique`). The published orders are the orders of the two symmetric groups, $|S_n| = n!$ (`Envelope.card_perm_fin`), they bracket the scale (`Envelope.group_envelope`), and the lower group embeds injectively in the upper one (`Envelope.perm_succ_embedding`), so the sandwich is a chain of groups and not merely a pair of numbers.*

5.3 The local field prime

With t_d and an initial word bound w_d fixed by a published policy from the bit length of $N(d)$, the field parameter is

$$p_d = \min\{p \text{ prime} : p > N(d), p \equiv 1 \pmod{2^{t_d}}, p < 2^{w_d}\},$$

with the deterministic escalation $w \mapsto w+1$, capped at the 64-bit word boundary, if the window is empty; the entry records the word bound actually used, and the check demands that an escalated bound was necessary. The congruence is the Goldilocks-style criterion: it is exactly what gives the field a multiplicative subgroup of order 2^{t_d} , the evaluation domain of an FFT/NTT-based commitment. The reference prime of that shape, $p_{\text{GL}} = 2^{64} - 2^{32} + 1 = 18446744069414584321$, is proved prime in the library (`Envelope.goldilocks_prime`) and carries a witness of order 2^{32} (`Envelope.goldilocks_root`).

Minimality is certified rather than asserted. The admissible candidates form the arithmetic progression $c_0, c_0 + 2^t, c_0 + 2 \cdot 2^t, \dots$ starting at the least integer above $N(d)$ congruent to 1 modulo 2^t (`Envelope.firstCandidate_least`), and the published *selection trace* exhibits, for every candidate the search stepped over, a nontrivial divisor of it.

Theorem 5.2 (selection; `Envelope.selection_sound`). *An accepted selection trace certifies that its endpoint is above $N(d)$, that 2^t divides its predecessor, and that no prime below it has those two properties.*

Theorem 5.3 (primality; `Pratt.prime_of_certifies`). *The published primality certificate is a Pratt chain: each step is either a number below `smallBound` settled by trial division, or a Lucas step whose factors of $n-1$ were established earlier in the chain. If the chain checker accepts a chain listing p , then p is prime.*

Theorem 5.4 (root of unity; `Envelope.orderOf_witness`). *Two modular exponentiations, $\omega^{2^t} = 1$ and $\omega^{2^{t-1}} \neq 1$, certify that the published witness has order exactly 2^t in $\mathbb{Z}/p_d$.*

5.4 What the reader checks, and what it means

`Envelope.Env.check` is a single Boolean check over the published fields: the two orders are the claimed factorials, they bracket $N(d)$, the parameters are the policy's, the selection trace runs from the first candidate to the published prime, the primality certificate accepts, the prime lies in the published window and the escalation was necessary, and the root-of-unity equations hold.

Theorem 5.5 (soundness; `Envelope.check_sound`). *If the check accepts then: the published orders are $|S_{n_d}|$ and $|S_{n_d+1}|$; $|G_d^-| \leq N(d) < |G_d^+|$; n_d is the unique index with that property; p_d is prime, exceeds $N(d)$, satisfies $2^{t_d} \mid p_d - 1$ and is the least prime with those two properties; $p_d < 2^{w_d} \leq 2^{64}$; the parameters are the policy's; and ω has order exactly 2^{t_d} .*

Theorem 5.6 (derivation; `Envelope.mkEnvelope_sound`). *The envelope is a function of $\kappa(d)$ alone, and the derivation emits nothing that does not pass the check. The independent verifier re-derives every published envelope from the published $\kappa(d)$ and compares.*

Theorem 5.7 (binding; `Envelope.commitment_binding`). *The envelope is serialised to machine words and committed with the length-prefixed Horner digest of the self-hosting layer; on that digest's binding domain two envelopes with the same commitment are equal. This is a theorem, not a cryptographic assumption. The commitment enters both the Fiat-Shamir challenge and the Merkle leaf, so the ledger root binds every envelope.*

5.5 This ledger

All 1570 of the 1570 envelopes pass the check and are reproduced by the derivation; 0 needed a word-bound escalation; the largest declaration-local prime is 14081. Each entry in the companion volume prints its own $\kappa(d)$, $N(d)$, group descriptors and orders, bracketing inequality, t_d , w_d , p_d , ω , the size of its selection trace and certificate, and the envelope commitment.

6 The verification recipe

A reader with a SHA-256 implementation and modular exponentiation can check any entry of the companion volume, in four steps.

1. **Statement.** Hash the printed statement; it must equal the printed statement digest.
2. **Envelope.** Run `Envelope.Env.check` on the printed envelope: the two factorials, the bracketing inequality, the policy parameters, the selection trace, the Pratt certificate, the window and the root-of-unity equations.
3. **Challenge.** Recompute

$$c = \text{SHA-256}(\text{harmonic} - \text{zk} - \text{ledger}/1 \parallel \text{name} \parallel \text{statement digest} \parallel \text{type digest} \parallel \text{env} \parallel \text{Com}(\text{envelope}) \parallel C \parallel t) \bmod q$$

and compare with the printed challenge.

4. **Equation.** Check $g^z h^w \equiv t C^c \pmod{p}$.
5. **Root.** Hash the entry's leaf line

$$\text{harmonic} - \text{zk} - \text{ledger}/1 \parallel \text{name} \parallel \text{statement digest} \parallel \text{type digest} \parallel \text{term digest} \parallel C \parallel t \parallel c \parallel z \parallel w \parallel \text{env} \parallel \text{Com}(\text{envelope}) \parallel N(d) \parallel n_d \parallel n_d! \parallel (n_d + 1)! \parallel p_d \parallel t_d \parallel w_d \parallel \omega$$

with the leaf tag `0x00`, and fold it with its siblings under the node tag `0x01` to reach the published root.

Steps 1–4 are exactly `Certificate.Entry.verify`; step 5 is `Merkle.verify`. The machine-readable form of the ledger, with the same numbers, is `artifacts/publication/ledger.jsonl`.

The five steps are also carried out mechanically by a second executable, `lake exe ledger_verify`, which reads that file and nothing else — no secret, no proof term, no part of the library beyond the publication layer — rebuilds each entry from the published fields, runs `Certificate.Entry.verify` on it, recomputes the Merkle root and compares it with the printed one. On the ledger of this document it reports all 1570 entries verified and the root reproduced.

7 What is claimed, and what is not

Claimed. The scheme is complete, extractable, hiding and zero-knowledge, in the precise senses of the theorems named above, all machine-checked in the library being published; every published entry verifies; the ledger root binds the ledger under collision resistance of SHA-256.

Not claimed.

1. *The transcripts do not certify that the proofs are correct.* A transcript proves knowledge of an opening of a commitment. It does not prove that the committed digest is the digest of a term the Lean kernel accepts for the printed statement. The evidence for that is the build: `lake build` succeeds, no `sorry` occurs, and the axiom closure of each declaration is printed in its entry. A reader who does not trust the issuer’s build has to obtain the withheld terms and replay them.
2. *The parameters are demonstration-scale.* At 51 bits the discrete logarithm is easy, so the binding property, and with it the soundness of the argument, has no practical force here; indeed the discrete logarithm y of g to base h is printed above. Hiding, being perfect and information-theoretic, is unaffected.
3. *The committed value is a residue.* The commitment is to the SHA-256 digest of the withheld serialisation *reduced modulo q* , so it binds 51 bits of that digest, not all 256.
4. *Collision resistance is assumed.* SHA-256 is implemented and its test vectors are checked, but no cryptographic property of it is proved; the Merkle binding theorem is stated conditionally, as the extraction of an explicit collision.
5. *Fiat–Shamir is used, not modelled.* What is proved is two-transcript extraction; the passage from that to non-interactive soundness is the usual random-oracle argument and is not formalised.
6. *The envelope is a size certificate, not a proof of correctness.* The group sandwich and the derived field prime are deterministic, non-truncated, auditable parameters of the declaration. They do not establish $\text{KernelChecks}(\text{env}_d, v_d : \tau_d)$: nothing in the envelope relates the withheld term to the published type. To carry semantic force the envelope would have to parameterise a circuit relation that checks the hidden term’s canonical parse, DAG constraints and kernel-checking trace against the pinned environment; that relation is not formalised here.
7. *Statement text is a pretty printer’s output.* The printed statement is what Lean’s delaborator produces under the stated options; the pretty-printer-independent commitment is the type digest, taken over the canonical serialisation of the elaborated type.

8 Three entries in full

The three entries below are the ledger’s records of a definition-level lemma, of the extraction theorem of the scheme itself, and of the theorem that says every entry verifies. They are reproduced verbatim from `paper/generated/sample-entries.tex`.

Publication.powMod_eq

theorem

$\forall (m \ e \ a : \mathbb{N}), \text{Publication.powMod } a \ e \ m = a^e \% m$

statement SHA-256 e93c949514e8ea2b948fbf58f4b2457e4c617f1c2debfa3196491666f7fc5b9

type SHA-256 efdd9c62a0d98c89adb7bad14eebd6a8d48f53a3705674aef3f51c7ef3f4f719

withheld term SHA-256 9287ca9c15a2f42e3b8f9d626468c1a7848a1559fa869e9fe4cc5fb3863ac168

withheld term: 19464 bytes, 1664 kernel nodes axioms: `propext`, `Classical.choice`, `Quot.sound`

$C = 1383996721334587$, $t = 989949616747978$, $c = 398258583735103$, $z = 634345468260876$, $w = 561408117353714$
 $\kappa(d) = (41, 1664, 138, 1, 6029)$, $N(d) = 7873$; $G^- = S_7$ of order 5040, $G^+ = S_{7+1}$ of order 40320, $5040 \leq 7873 < 40320$
 $p_d = 7937$ with $2^6 \mid p_d - 1$ and $p_d < 2^{21}$, $\omega = 1707$ of order 2^6 , 0 candidates skipped, 3 certificate steps
 envelope commitment 1715479396024838158636162781525979272059783486891126111421448618835599213036607447
 8139685497726073008026455641456484654830992867845841548705013970142526825021797182359899074943884433635068184
 3051227691876767484093046815282813395689452046034832698776633263945053164687130580912352930670171285963911309
 4270635587510458363433412668191418021496969885894895062063468411144571778771997416121566978721513965046285079
 4989885597877998176806992972496399510019052030709002163580728578828262991247194101011770958859829421363611133
 599721536635983689066400024183282860231240496772702743419706257520666085833122097125985653512787743595765297
 513274303082278158367

Publication.Ledger.special_soundness

theorem

$\forall \{G : \text{Type } u_1\} [\text{inst} : \text{CommGroup } G] \{q : \mathbb{N}\} [\text{NeZero } q] \{g \ h : G\} [\text{Fact } (\text{Nat.Prime } q)], g \wedge q = 1 \rightarrow h \wedge q = 1 \rightarrow \forall \{C : G\}, C \wedge q = 1 \rightarrow \forall (T \ T' : \text{Publication.Ledger. Transcript } G \ q), \text{Publication.Ledger.Accepts } g \ h \ C \ T \rightarrow \text{Publication.Ledger.Accepts } g \ h \ C \ T' \rightarrow T.t = T'.t \rightarrow T.c \neq T'.c \rightarrow \text{Publication.Ledger.com } g \ h \ (\text{Publication.Ledger.extract } T \ T').1 \ (\text{Publication.Ledger.extract } T \ T').2 = C$

statement SHA-256 60eaf91458bdfffabced5da8e406f64b922eba20035b150586a9ddd2dc14d529

type SHA-256 f4e11856bdafa9d77784633f9bf66c15167d01f778e75d8deac3ce265455d2d76

withheld term SHA-256 ea8ff217a06e130098280796f4e01f8b034ce434b01b42e6a654d2bce582df25

withheld term: 9610 bytes, 813 kernel nodes axioms: propext, Quot.sound, Classical.choice

$C = 566690819267306$, $t = 1031587586037941$, $c = 625339895132000$, $z = 902633265170063$, $w = 933414959521603$

$\kappa(d) = (234, 813, 90, 2, 2625)$, $N(d) = 3764$; $G^- = S_6$ of order 720, $G^+ = S_{6+1}$ of order 5040, $720 \leq 3764 < 5040$

$p_d = 4289$ with $2^6 \mid p_d - 1$ and $p_d < 2^{20}$, $\omega = 3213$ of order 2^6 , 8 candidates skipped, 3 certificate steps

envelope commitment 1285021588113498911367143862249967941539480832008502815762433960531256378897400005
 2058654919883829579794147903850008137500742721565996057213936315652191036021784895168936184001058650934916287
 1905869191454718972948146179417009232925999403001395364005458439976227799965162985856958816638735570104266603
 8349560124863097243327452429832800410291137967776347024953911788669394092524475751995702542314473036997769350
 7347470634824277125477426813920873990182209063733943033809519210755892418399964327728258269958437240106653315
 5042072583802086489723599023828019219313295120478266658553790748375937822204829524200392648067695747227216940
 3370772984200096716441251603849095454658603495381668144520408898505685071193677080233901781629181697708646997
 1702012959640095072987559984425808561321345643281602772445228250527945627028216880366303924079762110237271651
 0855178025528581845422185978995915334473295534385059329008436890958823491

Publication.Certificate.mkEntry_verifies

theorem

$\forall (\text{name statement typeDigest termDigest} : \text{String}) (\text{termBytes} : \mathbb{N}) (\text{axioms seed} : \text{String}) \{\text{env} : \text{Publication.Envelope.Env}\}, \text{env.check} = \text{true} \rightarrow (\text{Publication.Certificate.mkEntry name statement typeDigest termDigest termBytes axioms seed env}).\text{verify} = \text{true}$

statement SHA-256 6e75010821258cbf4176147501e1063892e454ff12fb4d2f25078dfc2dbb56a9

type SHA-256 6df458c30991410abbd96d0e94ded3ae9f8420c8135e3aea97d303f2569aa317

withheld term SHA-256 05a54efe6a7e92bfa8ae28458d2afbb36be14b4a455d3cb0cce89b3b354001bc

withheld term: 4610 bytes, 391 kernel nodes axioms: propext, Quot.sound, Classical.choice, Lean.ofReduceBool, Lean.trustCompiler

$C = 68111225781402$, $t = 24273351181621$, $c = 1012282009593115$, $z = 466972314298360$, $w = 235715936112355$

$\kappa(d) = (42, 391, 73, 1, 1134)$, $N(d) = 1641$; $G^- = S_6$ of order 720, $G^+ = S_{6+1}$ of order 5040, $720 \leq 1641 < 5040$

$p_d = 1697$ with $2^5 \mid p_d - 1$ and $p_d < 2^{19}$, $\omega = 69$ of order 2^5 , 1 candidates skipped, 1 certificate steps

envelope commitment 1890162730551844296133059116219761611989575226915589093181484040497948838314585278
 3659130120218400984113482050895589994830279434806736984841493606258641427360652031416340103552781127255172902
 6317397458266281386589080916648731031072683643252190741487253724273387714906064658200820305682859866637036415
 613026907859848527187271169282698259093154055824108806912412017230269468639232

9 The library, module by module

module	declarations	theorems	definitions	withheld bytes
RequestProject.CatalogMining	73	52	21	180918
RequestProject.ComplexityLattice	86	57	29	136513
RequestProject.ConstraintTower	23	16	7	43513
RequestProject.Elaboration	30	18	12	48202
RequestProject.Execution	13	6	7	32372
RequestProject.FormCertificates	26	10	16	43012
RequestProject.FormTower	92	64	28	168663
RequestProject.GlobalSandwich	53	47	6	111491
RequestProject.GradedReflection	55	41	14	279168
RequestProject.GroupForms	30	23	7	119472
RequestProject.GroupSandwich	54	48	6	295574
RequestProject.OrbifoldReflection	11	9	2	130475
RequestProject.ProofKernel	34	25	9	95577
RequestProject.Publication.Certificate	33	7	26	16190
RequestProject.Publication.Envelope	65	23	42	81483
RequestProject.Publication.FieldSelect	15	11	4	60329
RequestProject.Publication.Group	14	12	2	54382
RequestProject.Publication.Ledger	29	15	14	103641
RequestProject.Publication.Merkle	22	8	14	44340
RequestProject.Publication.Params	37	26	11	51816
RequestProject.Publication.Pratt	22	8	14	39038
RequestProject.Publication.Sandwich	19	8	11	13728
RequestProject.Publication.Sha256	18	4	14	22555
RequestProject.Reflection	55	32	23	123568
RequestProject.SHHS.Catalog	20	7	13	13424
RequestProject.SHHS.CircuitIR	45	16	29	101495
RequestProject.SHHS.Encoding	28	13	15	92891
RequestProject.SHHS.Holography	61	22	39	95541
RequestProject.SHHS.MetaCircuit	16	7	9	11114
RequestProject.SHHS.SelfDescription	22	17	5	27758
RequestProject.SHHS.SystemIR	39	10	29	29305
RequestProject.SelfCertificate	17	7	10	33454
RequestProject.SelfCertification	48	29	19	80969
RequestProject.SelfCircuit	26	15	11	83793
RequestProject.SelfReflection	35	26	9	58863
RequestProject.SelfZK	21	12	9	47879
RequestProject.Shadows	70	45	25	60848
RequestProject.TypeChecker	10	7	3	20318
RequestProject.ZKArgument	34	14	20	198629
RequestProject.ZKCircuit	51	29	22	192505
RequestProject.ZKFormCatalog	43	29	14	36715
RequestProject.ZKForms	27	16	11	73010
RequestProject.ZKMining	48	33	15	243684

10 Reproducing this document

1. `lake build` — builds the library, including the publication layer.
2. `lake exe ledger_export` — enumerates the library, builds every entry with the verified definitions, runs the verifier on all of them, and writes `artifacts/publication/` and `paper/generated/`.
3. `bash scripts/build-papers.sh` — typesets this document and the companion volume.

4. `lake exe ledger_verify` — optional: re-checks the published ledger from `artifacts/publication/` alone.

The pipeline is deterministic: the same repository yields the same digests, commitments, transcripts and root.